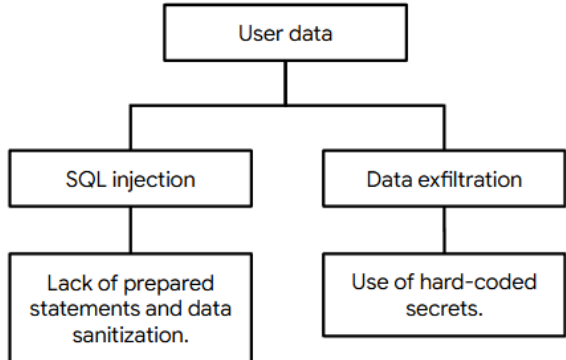


PASTA worksheet

Stages	Sneaker company
I. Define business and security objectives	<i>The app will process transactions, expected to be clear and quick to process. Therefore, it's necessary to consider regulations related to payment and transactions; must be in compliance with PCI-DSS.</i> <i>The app does a lot of back-end processing; it allows users to rate and contact sellers, also it must have a smooth process for login, signup and manage of accounts, and should process clear and quick transactions.</i>
II. Define the technical scope	List of technologies used by the application: <i>Application programming interface (API)</i> <i>Public key infrastructure (PKI) (AES and RSA)</i> <i>SHA-256</i> <i>SQL</i> First I would check the key size and versions used for the protocols of the PKI framework, for example RSA, expecting at least 2,048 of size. Then, I would check SQL; the version, configuration, and if the application is vulnerable to SQL injections. After that, I would check the other technologies. The process is done in this order because API, PKI and SHA-256 are standard protocols with very tested implementations and it should be unlikely to have vulnerabilities there, apart from the version or key size selected by the app developers.
III. Decompose application	Data flow diagram:  <pre> graph LR User[User] -- "Searching for sneakers for sale." --> Process((Product search process)) Process -- "Listings of current inventory." --> Database[Database] Database --> Process Process --> User </pre> The PKI framework is used to protect data in transit and at rest, in this case used to protect the communication between user, product search process and database, and in the latter to store

	protected data (same as SHA-256). SQL is used to manage the data contained in the database; this is done by the product search process querying the database. The API is provided by the product search process and probably it communicates with other third party APIs to do background operations.
IV. Threat analysis	2 Potential threats to the application: • Injection. A hacker could load malicious scripts into the application. • Data exfiltration. A disgruntled employee, hacker or malware could steal information from the database.
V. Vulnerability analysis	2 vulnerabilities that can be exploited by the threats listed before: • Use of hard-coded secrets. • Lack of prepared statements.
VI. Attack modeling	Attack tree:  <pre> graph TD A[User data] --> B[SQL injection] A --> C[Data exfiltration] B --> D[Lack of prepared statements and data sanitization.] C --> E[Use of hard-coded secrets.] </pre>
VII. Risk analysis and impact	Security controls that can reduce the chances of a security incident: • Secret managers. • MFA. • Firewalls. • Principle of least privilege.